Experiment no: - 2

(PART B: TO BE COMPLETED BY STUDENTS)

Experiment Name: - Data Manipulation Language statements

Aim: - Performing practical by using DML statements statements

Roll. No:C-60	Name:Avdhut Madhavi
Class : C	Batch: C2
Date of Experiment:	Date of Submission:
Grade:	

B.1: Read the given question carefully and write a SQL statement and check the output. (Put the query and its output after each question.)

After successfully creating the database and tables using **DDL statements (CREATE, ALTER, DROP)**, perform the following **DML (Data Manipulation Language) operations**:

1.INSERT into Movie_Details

```
INSERT INTO MOVIE_DETAILS
(movie_id, title, movie_genre, language, duration, rating)
VALUES
(101, 'Inception', 'Sci-Fi', 'English', 148, 8.8),
(102, 'Dangal', 'Drama', 'Hindi', 161, 8.3),
(103, 'Interstellar', 'Sci-Fi', 'English', 169, 8.7),
(104, '3 Idiots', 'Comedy', 'Hindi', 170, 8.4),
(105, 'Drishyam', 'Thriller', 'Hindi', 163, 8.2);

SELECT * FROM MOVIE_DETAILS;
```

Result Grid | Filter Rows: | Edit: | Export/Import:

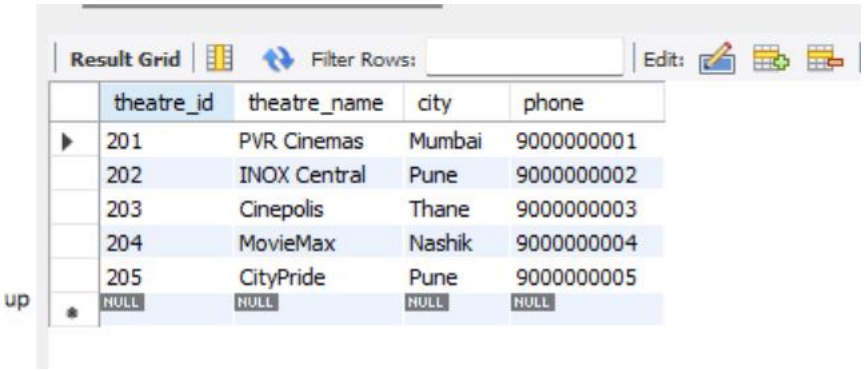
movie_id	title	movie_genre	language	duration	rating	remarks
101	Inception	Sci-Fi	English	148	8.8	NULL
102	Dangal	Drama	English	161	8.3	NULL
103	Interstellar	Sci-Fi	English	169	8.7	NULL
104	3 Idiots	Comedy	Hindi	170	8.4	NULL
105	Drishyam	Thriller	Hindi	163	8.2	NULL
NULL	NULL	NULL	NULL	NULL	NULL	NULL

2. INSERT into THEATRE

```
INSERT INTO THEATRE
(theatre_id, theatre_name, city, phone)
VALUES
(201, 'PVR Cinemas', 'Mumbai', '9000000001'),
(202, 'INOX Central', 'Pune', '9000000002'),
```

(203, 'Cinepolis', 'Thane', '9000000003'),
(204, 'MovieMax', 'Nashik', '9000000004'),
(205, 'CityPride', 'Pune', '9000000005');

SELECT * FROM THEATRE;


Result Grid | Filter Rows: | Edit:

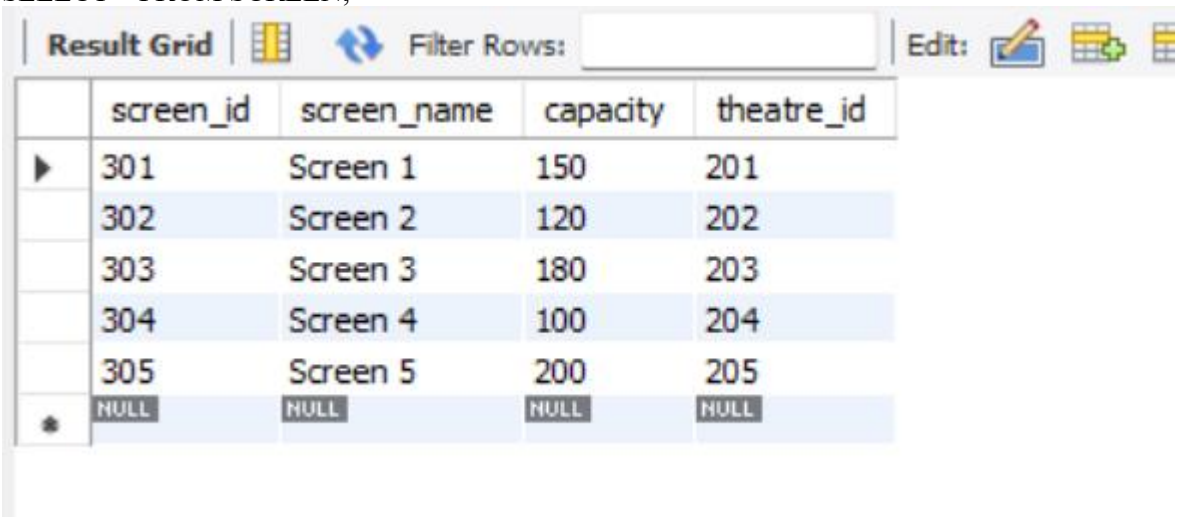
theatre_id	theatre_name	city	phone
201	PVR Cinemas	Mumbai	9000000001
202	INOX Central	Pune	9000000002
203	Cinepolis	Thane	9000000003
204	MovieMax	Nashik	9000000004
205	CityPride	Pune	9000000005
NULL	NULL	NULL	NULL

up

3. INSERT into SCREEN

INSERT INTO SCREEN
(screen_id, screen_name, capacity, theatre_id)
VALUES
(301, 'Screen 1', 150, 201),
(302, 'Screen 2', 120, 202),
(303, 'Screen 3', 180, 203),
(304, 'Screen 4', 100, 204),
(305, 'Screen 5', 200, 205);

SELECT * FROM SCREEN;


Result Grid | Filter Rows: | Edit:

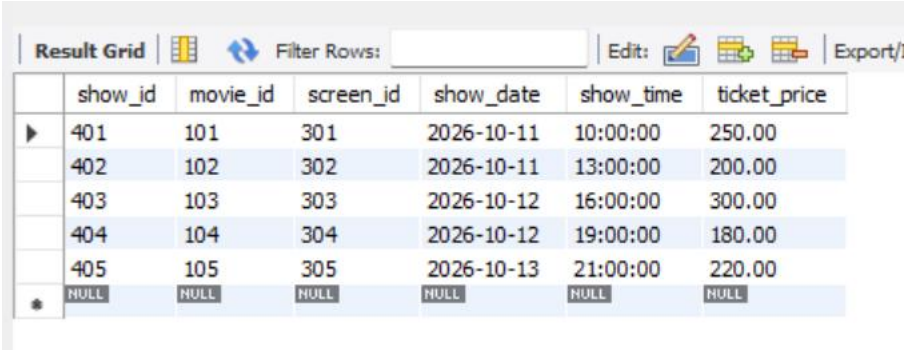
screen_id	screen_name	capacity	theatre_id
301	Screen 1	150	201
302	Screen 2	120	202
303	Screen 3	180	203
304	Screen 4	100	204
305	Screen 5	200	205
NULL	NULL	NULL	NULL

4. INSERT into SHOWTIME

INSERT INTO SHOWTIME
(show_id, movie_id, screen_id, show_date, show_time, ticket_price)
VALUES
(401, 101, 301, '2026-10-11', '10:00:00', 250.00),
(402, 102, 302, '2026-10-11', '13:00:00', 200.00),

(403, 103, 303, '2026-10-12', '16:00:00', 300.00),
(404, 104, 304, '2026-10-12', '19:00:00', 180.00),
(405, 105, 305, '2026-10-13', '21:00:00', 220.00);

SELECT * FROM SHOWTME;



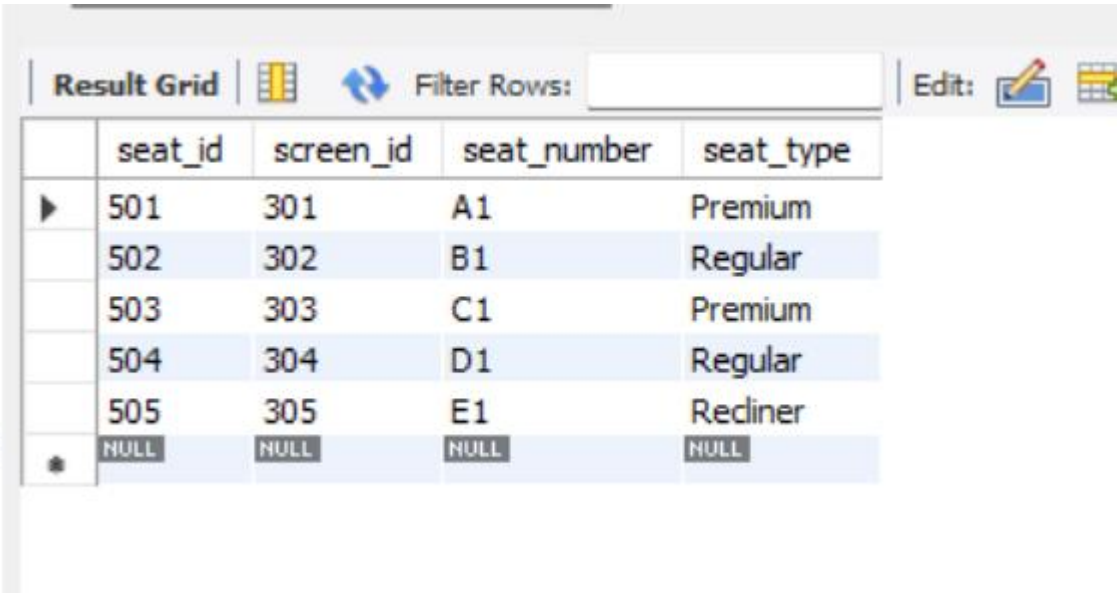
Result Grid | Filter Rows: | Edit: | Export/

show_id	movie_id	screen_id	show_date	show_time	ticket_price
401	101	301	2026-10-11	10:00:00	250.00
402	102	302	2026-10-11	13:00:00	200.00
403	103	303	2026-10-12	16:00:00	300.00
404	104	304	2026-10-12	19:00:00	180.00
405	105	305	2026-10-13	21:00:00	220.00
NULL	NULL	NULL	NULL	NULL	NULL

5. INSERT into SEAT

INSERT INTO SEAT
(seat_id, screen_id, seat_number, seat_type)
VALUES
(501, 301, 'A1', 'Premium'),
(502, 302, 'B1', 'Regular'),
(503, 303, 'C1', 'Premium'),
(504, 304, 'D1', 'Regular'),
(505, 305, 'E1', 'Recliner');

SELECT * FROM SEAT;



Result Grid | Filter Rows: | Edit:

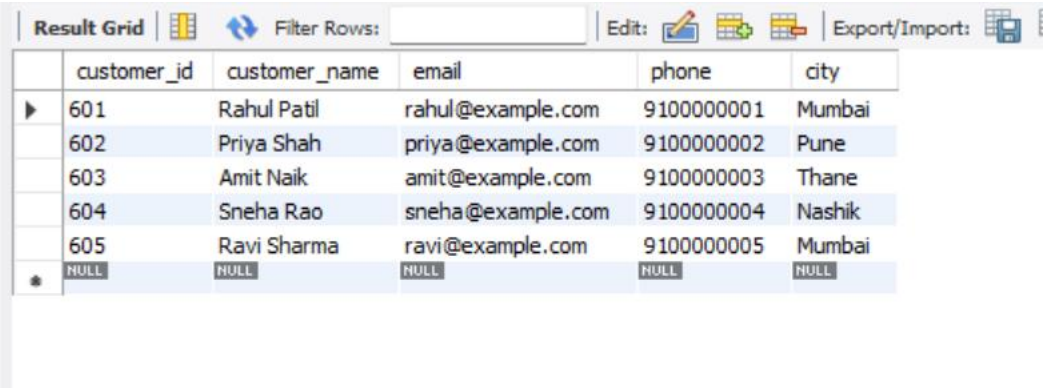
seat_id	screen_id	seat_number	seat_type
501	301	A1	Premium
502	302	B1	Regular
503	303	C1	Premium
504	304	D1	Regular
505	305	E1	Recliner
NULL	NULL	NULL	NULL

6. INSERT into CUSTOMER

INSERT INTO CUSTOMER
(customer_id, customer_name, email, phone, city)
VALUES

(601, 'Rahul Patil', 'rahul@example.com', '9100000001', 'Mumbai'),
(602, 'Priya Shah', 'priya@example.com', '9100000002', 'Pune'),
(603, 'Amit Naik', 'amit@example.com', '9100000003', 'Thane'),
(604, 'Sneha Rao', 'sneha@example.com', '9100000004', 'Nashik'),
(605, 'Ravi Sharma', 'ravi@example.com', '9100000005', 'Mumbai');

SELECT * FROM CUSTOMER;


Result Grid | Filter Rows: | Edit: | Export/Import:

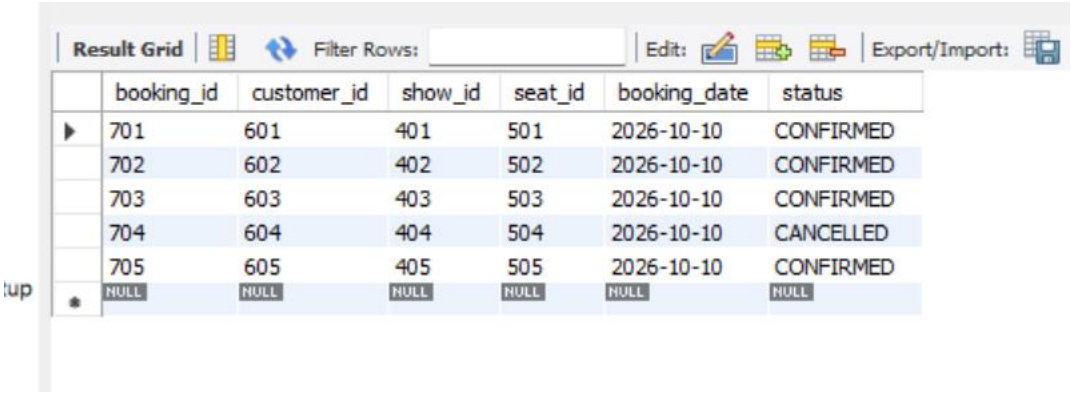
customer_id	customer_name	email	phone	city
601	Rahul Patil	rahul@example.com	9100000001	Mumbai
602	Priya Shah	priya@example.com	9100000002	Pune
603	Amit Naik	amit@example.com	9100000003	Thane
604	Sneha Rao	sneha@example.com	9100000004	Nashik
605	Ravi Sharma	ravi@example.com	9100000005	Mumbai
NULL	NULL	NULL	NULL	NULL

7. Create and Insert into BOOKINGS

```
CREATE TABLE booking (
   booking_id INT PRIMARY KEY,
   customer_id INT,
   show_id INT,
   seat_id INT,
   booking_date DATE DEFAULT (CURRENT_DATE),
   status VARCHAR(15) DEFAULT 'CONFIRMED',
   FOREIGN KEY (customer_id)
      REFERENCES customer(customer_id),
   FOREIGN KEY (show_id)
      REFERENCES showtime(show_id),
   FOREIGN KEY (seat_id)
      REFERENCES seat(seat_id)
);

INSERT INTO booking
(booking_id, customer_id, show_id, seat_id, booking_date, status)
VALUES
(701, 601, 401, 501, '2026-10-10', 'CONFIRMED'),
(702, 602, 402, 502, '2026-10-10', 'CONFIRMED'),
(703, 603, 403, 503, '2026-10-10', 'CONFIRMED'),
(704, 604, 404, 504, '2026-10-10', 'CANCELLED'),
(705, 605, 405, 505, '2026-10-10', 'CONFIRMED');

SELECT * FROM BOOKING;
```



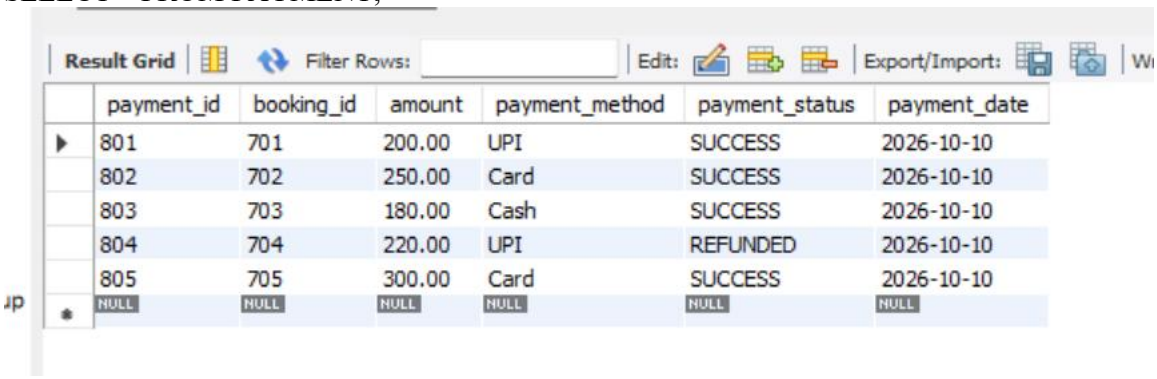
Result Grid | Filter Rows: | Edit: | Export/Import:

booking_id	customer_id	show_id	seat_id	booking_date	status
701	601	401	501	2026-10-10	CONFIRMED
702	602	402	502	2026-10-10	CONFIRMED
703	603	403	503	2026-10-10	CONFIRMED
704	604	404	504	2026-10-10	CANCELLED
705	605	405	505	2026-10-10	CONFIRMED
NULL	NULL	NULL	NULL	NULL	NULL

8. Create and Insert into PAYMENT:

```
CREATE TABLE payment (
   payment_id INT PRIMARY KEY,
   booking_id INT,
   amount DECIMAL(8,2),
   payment_method VARCHAR(20),
   payment_status VARCHAR(15),
   payment_date DATE DEFAULT (CURRENT_DATE),
   FOREIGN KEY (booking_id)
      REFERENCES booking(booking_id)
);
INSERT INTO payment
(payment_id, booking_id, amount, payment_method, payment_status, payment_date)
VALUES
(801, 701, 200.00, 'UPI', 'SUCCESS', '2026-10-10'),
(802, 702, 250.00, 'Card', 'SUCCESS', '2026-10-10'),
(803, 703, 180.00, 'Cash', 'SUCCESS', '2026-10-10'),
(804, 704, 220.00, 'UPI', 'REFUNDED', '2026-10-10'),
(805, 705, 300.00, 'Card', 'SUCCESS', '2026-10-10');

SELECT * FROM PAYMENT;
```



Result Grid | Filter Rows: | Edit: | Export/Import:

payment_id	booking_id	amount	payment_method	payment_status	payment_date
801	701	200.00	UPI	SUCCESS	2026-10-10
802	702	250.00	Card	SUCCESS	2026-10-10
803	703	180.00	Cash	SUCCESS	2026-10-10
804	704	220.00	UPI	REFUNDED	2026-10-10
805	705	300.00	Card	SUCCESS	2026-10-10
NULL	NULL	NULL	NULL	NULL	NULL

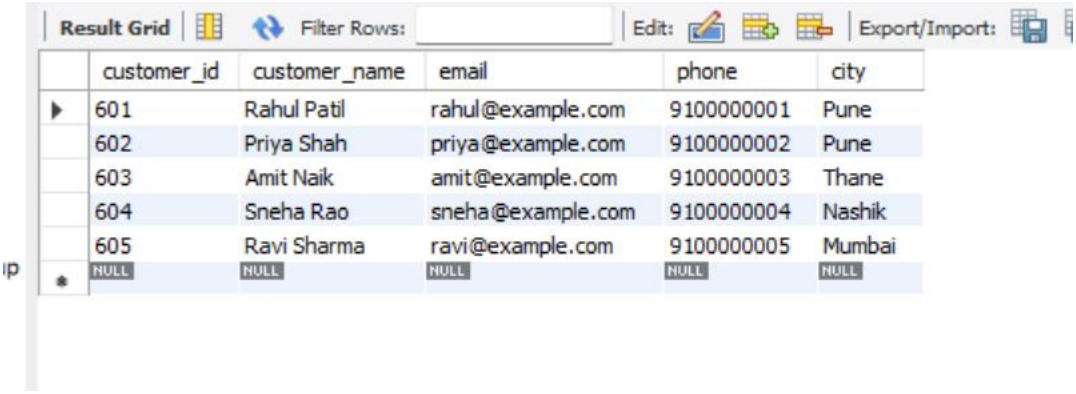
2. UPDATE queries

1. Customer_Details04
A. Update a customer's city
Change the city of customer 601 to Pune.

```
UPDATE customer
SET city = 'Pune'
```

WHERE customer_id = 601;

SELECT * FROM customer;

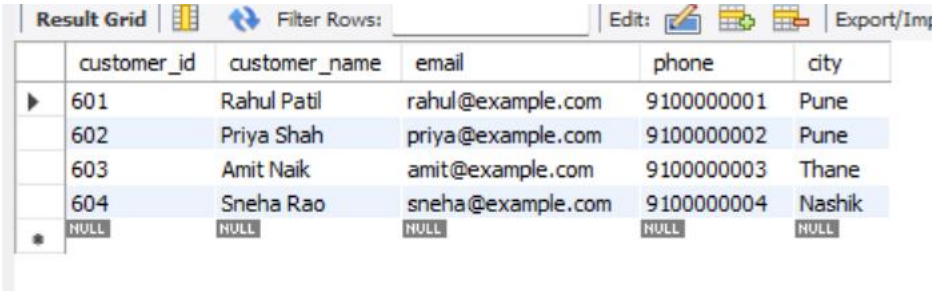


customer_id	customer_name	email	phone	city
601	Rahul Patil	rahul@example.com	9100000001	Pune
602	Priya Shah	priya@example.com	9100000002	Pune
603	Amit Naik	amit@example.com	9100000003	Thane
604	Sneha Rao	sneha@example.com	9100000004	Nashik
605	Ravi Sharma	ravi@example.com	9100000005	Mumbai
NULL	NULL	NULL	NULL	NULL

B. DELETE query
A. Delete a specific customer

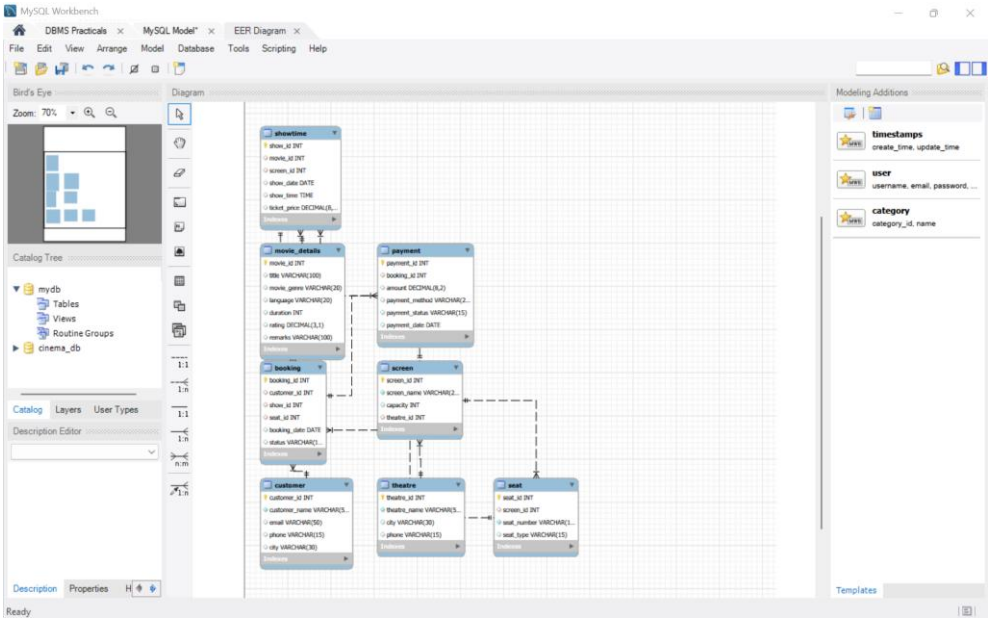
DELETE FROM customer
WHERE customer_id = 605;

SELECT * FROM CUSTOMER;



customer_id	customer_name	email	phone	city
601	Rahul Patil	rahul@example.com	9100000001	Pune
602	Priya Shah	priya@example.com	9100000002	Pune
603	Amit Naik	amit@example.com	9100000003	Thane
604	Sneha Rao	sneha@example.com	9100000004	Nashik
NULL	NULL	NULL	NULL	NULL

UPDATED EER DIAGRAM:



B.2: Write an observation according to the query execution.

Observation: **Cinema Ticket Booking System**

1. Created the required tables for the Cinema Ticket Booking System using DDL statements.
2. Inserted records into the tables: movie_details, theatre, screen, showtime, seat, customer, booking, and payment.
3. Retrieved table records successfully using SELECT statements.
4. Updated customer details and booking statuses using UPDATE statements.
5. Deleted selected records using DELETE statements.
6. Applied primary-key and foreign-key constraints to maintain data integrity and relationships between tables.
7. Used constraints to prevent duplicate primary keys and invalid foreign-key values.
8. Observed how foreign-key relationships affect the deletion of records in related tables.

B.3: Write an answer of the following questions

1.What is the difference between **DDL and DML**?

DDL (Data Definition Language)	**DML (Data Manipulation Language)**
Used to create, modify, and delete database structures.	Used to insert, update, and delete records in tables.
Deals with tables and other database objects.	Deals with data stored in tables.
Changes the database structure.	Changes the data stored in the database.
Examples: CREATE, ALTER, DROP, TRUNCATE.	Examples: INSERT, UPDATE, DELETE.

2.What is the difference between DELETE, TRUNCATE, and DROP in terms of performance?

DELETE	**TRUNCATE**	**DROP**
Removes selected rows or all rows from a table.	Removes all rows from a table.	Removes the entire table.
Supports the WHERE clause.	Does not support the WHERE clause.	Does not support the WHERE clause.

Generally slower for removing all rows.	Generally faster for removing all rows.	Removes both data and table structure.
Keeps the table structure.	Keeps the table structure.	Deletes the table structure and data.
In MySQL, an InnoDB DELETE can be rolled back before committing the transaction.	Generally cannot be rolled back in MySQL because it causes an implicit commit.	Generally cannot be rolled back in MySQL because it causes an implicit commi